

ERC-4626 Yield Vault with MCDM Scoring

Sergei Solovev

sesesolovev@edu.hse.ru

HSE University, April 2026

Project 1: Custom DeFi Protocol

What We Built

Problem: DeFi lending rates (Aave, Compound) change every block. Manual rebalancing is expensive and error-prone.

Solution: An ERC-4626 vault that automatically moves user funds to the best-performing protocol based on multi-factor scoring.

How it works:

Step	What happens
1	User deposits USDC, receives <code>aiUSDC</code> shares
2	Off-chain agent reads rates from Aave V3 and Compound V3
3	Agent scores each protocol (APY, risk, cost, stability)
4	If score delta > threshold, agent signs a rebalance with EIP-712
5	Vault verifies signature on-chain and executes the switch
6	If agent goes offline > 6h, Chainlink Automation takes over

Architecture

```
+-----+
| Off-chain: Python Agent (runs every hour) |
| 1. Read APY, utilization, TVL, gas price |
| 2. Smooth rates with EMA                 |
| 3. Score protocols with MCDM (4 factors)  |
| 4. Sign decision with EIP-712 typed data |
+-----+
|                                     |
|               | signed tx          |
|               |                   |
+-----+
| On-chain: AIVault.sol (ERC-4626 + UUPS proxy) |
| - Verify keeper signature (ECDSA)             |
| - Check nonce, timestamp, cooldown            |
| - Execute rebalance via adapter               |
| +-- AaveV3Adapter      +-- CompoundV3Adapter |
+-----+
| Fallback: Chainlink Automation (if agent offline > 6h) |
```

Key contracts: AIVault.sol, StrategyManager.sol, AaveV3Adapter.sol, CompoundV3Adapter.sol, RateMath.sol

Proxy pattern: UUPS (ERC-1967) for upgradeability

Key Formulas

ERC-4626 share price (with inflation attack protection):

$$s = \left\lfloor \frac{a \cdot (S + 10^6)}{A + 1} \right\rfloor$$

APY normalization (cross-protocol):

- Aave: $APY = \text{liquidityRate}_{RAY} / 10^9$
- Compound: $APY = r_{sec} \times 31,557,600$

EMA smoothing (anti-manipulation): $S_t = 0.3 \cdot R_t + 0.7 \cdot S_{t-1}$

MCDM scoring model:

$$\text{Score}_i = 0.40 \cdot f_{APY} + 0.25 \cdot f_{Risk} + 0.20 \cdot f_{Cost} + 0.15 \cdot f_{Stability}$$

Rebalance if: $\text{Score}_{best} - \text{Score}_{current} \geq 0.05$

Why multi-factor? Example: Aave has 6% APY but 95% utilization (risky). Compound has 5% APY but 30% utilization (safe). MCDM picks Compound. Simple APY comparison picks Aave (wrong).

Security and Testing

Security measures:

Threat	Protection
Inflation attack	Virtual shares (10^6 offset)
Reentrancy	ReentrancyGuard on all external functions
Rate manipulation	EMA + 5% jump guard
Signature forgery	EIP-712 domain + ECDSA verification
Replay attack	Sequential nonce + 5-min timestamp TTL
Agent downtime	Chainlink Automation fallback (6h)
Rapid exploitation	1-hour cooldown between rebalances

Testing: 67 tests, 0 failures

Category	Tests	Method
Unit (Solidity)	37	Concrete + fuzz (1000 runs each)
Integration	4	Full lifecycle (deposit -> rebalance -> yield -> withdraw)

Deployment on Sepolia

Network: Ethereum Sepolia (Chain ID: 11155111)

Deployed contracts:

Contract	Address
AaveV3Adapter	0x8545D79f6FaB51EDc93Cf024fBD1FfAc98504ba1
CompoundV3Adapter	0xEB0D41F07691765314B9A45645Ee995d879c7ac7
StrategyManager	0x353469534dA4FB64d52Ae5059CEFd098557eBFa9
AlVault (proxy)	0x1324238b6F56Ccc785fC7f79Ca693546236Ad02C

All verified on **Sourcify** (exact match).

Requirements checklist:

- [x] Solidity implementation (6 contracts + 2 libraries)
- [x] GitHub repository in Foundry format
- [x] Source code, tests, deployment scripts

Thank You

Questions?

GitHub: `github.com/SergeySolovyev/ai-yield-vault`

Contact: sesesolovev@edu.hse.ru | @Sergey_Solovjov | www.sergeisolovev.com

Testnet: Ethereum Sepolia

Stack: Solidity 0.8.24, Python 3.12, Foundry, OpenZeppelin, Chainlink